Requirements for Outputs

All Excel files

Zero Formula Errors
- Every Excel model MUST be delivered with ZERO formula errors (#REF!, #DIV/0!, #VALUE!, #N/A, #NAME?)

Preserve Existing Templates (when updating templates)
- Study and EXACTLY match existing format, style, and conventions when modifying files
- Never impose standardized formatting on files with established patterns
- Existing template conventions ALWAYS override these guidelines

Financial models

Color Coding Standards
Unless otherwise stated by the user or existing template

Industry-Standard Color Conventions
- **Blue text (RGB: 0,0,255)**: Hardcoded inputs, and numbers users will change for scenarios
- **Black text (RGB: 0,0,0)**: ALL formulas and calculations
- **Green text (RGB: 0,128,0)**: Links pulling from other worksheets within same workbook
- **Red text (RGB: 255,0,0)**: External links to other files
- **Yellow background (RGB: 255,255,0)**: Key assumptions needing attention or cells that need to be updated

Number Formatting Standards

Required Format Rules
- **Years**: Format as text strings (e.g., "2024" not "2,024")
- **Currency**: Use $#,##0 format; ALWAYS specify units in headers ("Revenue ($mm)")
- **Zeros**: Use number formatting to make all zeros "-", including percentages (e.g., "$#,##0;($#,##0);-")
- **Percentages**: Default to 0.0% format (one decimal)
- **Multiples**: Format as 0.0x for valuation multiples (EV/EBITDA, P/E)
- **Negative numbers**: Use parentheses (123) not minus -123

Formula Construction Rules

Assumptions Placement
- Place ALL assumptions (growth rates, margins, multiples, etc.) in separate assumption cells
- Use cell references instead of hardcoded values in formulas
- Example: Use =B5*(1+B6) instead of =B5*1.05

Formula Error Prevention
- Verify all cell references are correct
- Check for off-by-one errors in ranges
- Ensure consistent formulas across all projection periods
- Test with edge cases (zero values, negative numbers)
- Verify no unintended circular references

Documentation Requirements for Hardcodes
- Comment or in cells beside (if end of table). Format: "Source: [System/Document], [Date], [Specific Reference], [URL if applicable]"
- Examples:
- "Source: Company 10-K, FY2024, Page 45, Revenue Note, [SEC EDGAR URL]"
- "Source: Company 10-Q, Q2 2025, Exhibit 99.1, [SEC EDGAR URL]"
- "Source: Bloomberg Terminal, 8/15/2025, AAPL US Equity"
- "Source: FactSet, 8/20/2025, Consensus Estimates Screen"

XLSX creation, editing, and analysis

Overview

A user may ask you to create, edit, or analyze the contents of an .xlsx file. You have different tools and workflows available for different tasks.

Important Requirements

LibreOffice Required for Formula Recalculation: You can assume LibreOffice is installed for recalculating formula values using the `recalc.py` script. The script automatically configures LibreOffice on first run

Reading and analyzing data

Data analysis with pandas
For data analysis, visualization, and basic operations, use **pandas** which provides powerful data manipulation capabilities:

```python
import pandas as pd

# Read Excel
df = pd.read_excel('file.xlsx') # Default: first sheet
all_sheets = pd.read_excel('file.xlsx', sheet_name=None) # All sheets as dict

# Analyze
df.head() # Preview data
df.info() # Column info
df.describe() # Statistics

# Write Excel
df.to_excel('output.xlsx', index=False)
```

Excel File Workflows

CRITICAL: Use Formulas, Not Hardcoded Values

Always use Excel formulas instead of calculating values in Python and hardcoding them. This ensures the spreadsheet remains dynamic and updateable.

■ WRONG - Hardcoding Calculated Values
```python
# Bad: Calculating in Python and hardcoding result
total = df['Sales'].sum()
sheet['B10'] = total # Hardcodes 5000
```

```
# Bad: Computing growth rate in Python
growth = (df.iloc[-1]['Revenue'] - df.iloc[0]['Revenue']) / df.iloc[0]['Revenue']
sheet['C5'] = growth # Hardcodes 0.15

# Bad: Python calculation for average
avg = sum(values) / len(values)
sheet['D20'] = avg # Hardcodes 42.5
```

■ CORRECT - Using Excel Formulas
```python
# Good: Let Excel calculate the sum
sheet['B10'] = '=SUM(B2:B9)'

# Good: Growth rate as Excel formula
sheet['C5'] = '=(C4-C2)/C2'

# Good: Average using Excel function
sheet['D20'] = '=AVERAGE(D2:D19)'
```

This applies to ALL calculations - totals, percentages, ratios, differences, etc. The spreadsheet should be able to recalculate when source data changes.

Common Workflow
1. **Choose tool**: pandas for data, openpyxl for formulas/formatting
2. **Create/Load**: Create new workbook or load existing file
3. **Modify**: Add/edit data, formulas, and formatting
4. **Save**: Write to file
5. **Recalculate formulas (MANDATORY IF USING FORMULAS)**: Use the recalc.py script
```bash
python recalc.py output.xlsx
```
6. **Verify and fix any errors**:
- The script returns JSON with error details
- If `status` is `errors_found`, check `error_summary` for specific error types and locations
- Fix the identified errors and recalculate again
- Common errors to fix:
- `#REF!`: Invalid cell references
- `#DIV/0!`: Division by zero
- `#VALUE!`: Wrong data type in formula
- `#NAME?`: Unrecognized formula name

Creating new Excel files

```python
# Using openpyxl for formulas and formatting
from openpyxl import Workbook
from openpyxl.styles import Font, PatternFill, Alignment

wb = Workbook()
sheet = wb.active

# Add data
sheet['A1'] = 'Hello'
sheet['B1'] = 'World'
sheet.append(['Row', 'of', 'data'])
```

```
# Add formula
sheet['B2'] = '=SUM(A1:A10)'

# Formatting
sheet['A1'].font = Font(bold=True, color='FF0000')
sheet['A1'].fill = PatternFill('solid', start_color='FFFF00')
sheet['A1'].alignment = Alignment(horizontal='center')

# Column width
sheet.column_dimensions['A'].width = 20

wb.save('output.xlsx')
```

Editing existing Excel files

```python
# Using openpyxl to preserve formulas and formatting
from openpyxl import load_workbook

# Load existing file
wb = load_workbook('existing.xlsx')
sheet = wb.active # or wb['SheetName'] for specific sheet

# Working with multiple sheets
for sheet_name in wb.sheetnames:
sheet = wb[sheet_name]
print(f"Sheet: {sheet_name}")

# Modify cells
sheet['A1'] = 'New Value'
sheet.insert_rows(2) # Insert row at position 2
sheet.delete_cols(3) # Delete column 3

# Add new sheet
new_sheet = wb.create_sheet('NewSheet')
new_sheet['A1'] = 'Data'

wb.save('modified.xlsx')
```

Recalculating formulas

Excel files created or modified by openpyxl contain formulas as strings but not calculated values. Use the provided `recalc.py` script to recalculate formulas:

```bash
python recalc.py <excel_file> [timeout_seconds]
```

Example:
```bash
python recalc.py output.xlsx 30
```

The script:

- Automatically sets up LibreOffice macro on first run
- Recalculates all formulas in all sheets
- Scans ALL cells for Excel errors (#REF!, #DIV/0!, etc.)
- Returns JSON with detailed error locations and counts
- Works on both Linux and macOS

Formula Verification Checklist

Quick checks to ensure formulas work correctly:

Essential Verification
- [ ] **Test 2-3 sample references**: Verify they pull correct values before building full model
- [ ] **Column mapping**: Confirm Excel columns match (e.g., column 64 = BL, not BK)
- [ ] **Row offset**: Remember Excel rows are 1-indexed (DataFrame row 5 = Excel row 6)

Common Pitfalls
- [ ] **NaN handling**: Check for null values with `pd.notna()`
- [ ] **Far-right columns**: FY data often in columns 50+
- [ ] **Multiple matches**: Search all occurrences, not just first
- [ ] **Division by zero**: Check denominators before using `/` in formulas (#DIV/0!)
- [ ] **Wrong references**: Verify all cell references point to intended cells (#REF!)
- [ ] **Cross-sheet references**: Use correct format (Sheet1!A1) for linking sheets

Formula Testing Strategy
- [ ] **Start small**: Test formulas on 2-3 cells before applying broadly
- [ ] **Verify dependencies**: Check all cells referenced in formulas exist
- [ ] **Test edge cases**: Include zero, negative, and very large values

Interpreting recalc.py Output
The script returns JSON with error details:
```json
{
"status": "success", // or "errors_found"
"total_errors": 0, // Total error count
"total_formulas": 42, // Number of formulas in file
"error_summary": { // Only present if errors found
"#REF!": {
"count": 2,
"locations": ["Sheet1!B5", "Sheet1!C10"]
}
}
}
```

Best Practices

Library Selection
- **pandas**: Best for data analysis, bulk operations, and simple data export
- **openpyxl**: Best for complex formatting, formulas, and Excel-specific features

Working with openpyxl
- Cell indices are 1-based (row=1, column=1 refers to cell A1)
- Use `data_only=True` to read calculated values: `load_workbook('file.xlsx', data_only=True)`
- **Warning**: If opened with `data_only=True` and saved, formulas are replaced with values and permanently lost
- For large files: Use `read_only=True` for reading or `write_only=True` for writing
- Formulas are preserved but not evaluated - use recalc.py to update values

Working with pandas
- Specify data types to avoid inference issues: `pd.read_excel('file.xlsx', dtype={'id': str})`
- For large files, read specific columns: `pd.read_excel('file.xlsx', usecols=['A', 'C', 'E'])`
- Handle dates properly: `pd.read_excel('file.xlsx', parse_dates=['date_column'])`

Code Style Guidelines
IMPORTANT: When generating Python code for Excel operations:
- Write minimal, concise Python code without unnecessary comments
- Avoid verbose variable names and redundant operations
- Avoid unnecessary print statements

For Excel files themselves:
- Add comments to cells with complex formulas or important assumptions
- Document data sources for hardcoded values
- Include notes for key calculations and model sections